

Name: Dahiwal Gajanan Manikrao
Roll No: EN24107029
Class: SY - AI \& DS
Subject: Python for Visual AI

Assignment 9

An organisation wants to automate daily reports from CSV files. Automate data loading & cleaning, Generate summary reports, Save output as CSV / PDF, and schedule tasks using Python (cron / task scheduler)

Required Libraries

In [24]:

```
import pandas as pd
import matplotlib.pyplot as plt
from datetime import datetime
!pip install fpdf
from fpdf import FPDF
```

Requirement already satisfied: fpdf in /usr/local/lib/python3.12/dist-packages (1.7.2)

Loading Dataset

In [5]:

```
df = pd.read_csv("/content/sales_data_sample.csv", encoding="latin1")
df.head()
```

Out[5]:

	ORDERNUMBER	QUANTITYORDERED	PRICEEACH	ORDERLINENUMBER	SALES	ORDERDATE	ST
0	10107	30	95.70	2	2871.00	2/24/2003 0:00	SI
1	10121	34	81.35	5	2765.90	5/7/2003 0:00	SI
2	10134	41	94.74	2	3884.34	7/1/2003 0:00	SI
3	10145	45	83.26	6	3746.70	8/25/2003 0:00	SI
4	10159	49	100.00	14	5205.27	10/10/2003 0:00	SI

5 rows × 25 columns

Data Cleaning/Preprocessing

In [6]:

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2823 entries, 0 to 2822
Data columns (total 25 columns):
#   Column                Non-Null Count  Dtype
---  -
0   ORDERNUMBER           2823 non-null   int64
1   QUANTITYORDERED       2823 non-null   int64
2   PRICEEACH             2823 non-null   float64
3   ORDERLINENUMBER       2823 non-null   int64
4   SALES                 2823 non-null   float64
5   ORDERDATE             2823 non-null   object
6   STATUS                2823 non-null   object
7   QTR_ID               2823 non-null   int64
8   MONTH_ID             2823 non-null   int64
9   YEAR_ID              2823 non-null   int64
10  PRODUCTLINE          2823 non-null   object
11  MSRP                 2823 non-null   int64
12  PRODUCTCODE          2823 non-null   object
13  CUSTOMERNAME         2823 non-null   object
14  PHONE                2823 non-null   object
15  ADDRESSLINE1         2823 non-null   object
16  ADDRESSLINE2         302 non-null    object
17  CITY                 2823 non-null   object
18  STATE                1337 non-null   object
19  POSTALCODE           2747 non-null   object
20  COUNTRY              2823 non-null   object
21  TERRITORY            1749 non-null   object
22  CONTACTLASTNAME      2823 non-null   object
23  CONTACTFIRSTNAME     2823 non-null   object
24  DEALSIZE             2823 non-null   object
dtypes: float64(2), int64(7), object(16)
memory usage: 551.5+ KB
```

In [7]:

```
df.shape
```

Out[7]:

(2823, 25)

In [8]:

```
df.isnull().sum()
```

Out[8]:

	0
ORDERNUMBER	0
QUANTITYORDERED	0
PRICEEACH	0
ORDERLINENUMBER	0
SALES	0
ORDERDATE	0
STATUS	0

	0
QTR_ID	0
MONTH_ID	0
YEAR_ID	0
PRODUCTLINE	0
MSRP	0
PRODUCTCODE	0
CUSTOMERNAME	0
PHONE	0
ADDRESSLINE1	0
ADDRESSLINE2	2521
CITY	0
STATE	1486
POSTALCODE	76
COUNTRY	0
TERRITORY	1074
CONTACTLASTNAME	0
CONTACTFIRSTNAME	0
DEALSIZE	0

dtype: int64

In [11]:

```
df.drop_duplicates(inplace=True)
```

In [13]:

```
df.fillna(0, inplace=True)
```

In [14]:

```
df.isnull().sum()
```

Out[14]:

	0
ORDERNUMBER	0
QUANTITYORDERED	0
PRICEEACH	0
ORDERLINENUMBER	0
SALES	0
ORDERDATE	0
STATUS	0

	0
QTR_ID	0
MONTH_ID	0
YEAR_ID	0
PRODUCTLINE	0
MSRP	0
PRODUCTCODE	0
CUSTOMERNAME	0
PHONE	0
ADDRESSLINE1	0
ADDRESSLINE2	0
CITY	0
STATE	0
POSTALCODE	0
COUNTRY	0
TERRITORY	0
CONTACTLASTNAME	0
CONTACTFIRSTNAME	0
DEALSIZE	0

dtype: int64

In [16]:

```
df["ORDERDATE"] = pd.to_datetime(df["ORDERDATE"], errors="coerce")
```

Total Sales

$$Total\ Sales = \sum_{i=1}^n Sales_i$$

Total Orders

$$TotalOrders = n$$

In [18]:

```
total_sales = df["SALES"].sum()
total_orders = df.shape[0]
```

Estimated Profit

$$Total\ Profit = Total\ Sales \times r$$

where r is the profit ratio (e.g., 0.10).

In [19]:

```
total_profit = total_sales * 0.10
```

Sales by Country

$$\text{Sales}_{\text{country}} = \sum \text{Sales grouped by country}$$

In [20]:

```
country_sales = df.groupby("COUNTRY")["SALES"].sum()
```

Graphical Representation

Bar chart plotted using matplotlib to compare country-wise sales distribution

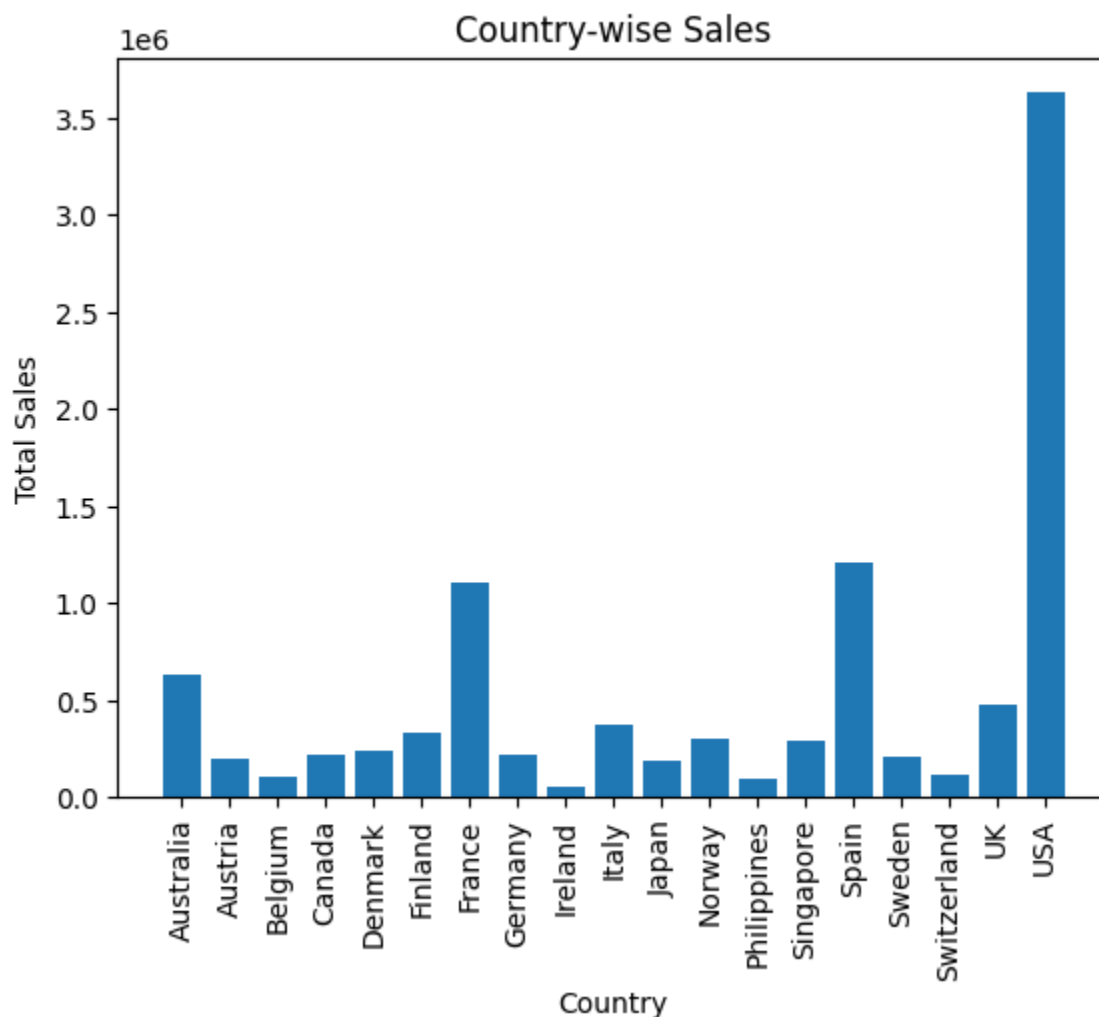
In [25]:

```
plt.figure()
plt.bar(country_sales.index, country_sales.values)

plt.xlabel("Country")
plt.ylabel("Total Sales")
plt.title("Country-wise Sales")

plt.xticks(rotation=90)

plt.show()
```



Summary Calculation

A dictionary is created to store key performance metrics of the dataset:

Total Orders : Number of records in the dataset

Total Sales : Sum of all sales values, rounded to 2 decimal places

Total Profit(Estimated) : Calculated profit based on total sales

Summary =

$$\begin{cases} \text{Total Orders} = n \\ \text{Total Sales} = \text{round}(\sum \text{Sales}, 2) \\ \text{Total Profit} = \text{round}(\text{Total Sales} \times r, 2) \end{cases}$$

The summary is then displayed using the print function.

In [21]:

```
summary = {
    "Total Orders": total_orders,
    "Total Sales": round(total_sales, 2),
    "Total Profit (Estimated)": round(total_profit, 2)
}
```

```
print(summary)
```

```
{'Total Orders': 2823, 'Total Sales': np.float64(10032628.85), 'Total Profit (Estimated)': np.float64(1003262.88)}
```

CSV Report Generation

The current date and time is obtained and formatted to create a unique file name:

```
date = datetime.now().strftime("
```

The summary dictionary is converted into a DataFrame:

```
summary_df = DataFrame(summary)
```

A CSV file name is generated using the formatted date:

```
csv_name = "sales_summary_" + date + ".csv"
```

Finally, the DataFrame is saved as a CSV file:

```
summary_df.to_csv(csv_name, index=False)
```

A confirmation message is displayed after saving the file.

In [22]:

```
date = datetime.now().strftime("%Y%m%d_%H%M")

summary_df = pd.DataFrame(list(summary.items()), columns=["Metric", "Value"])

csv_name = f"sales_summary_{date}.csv"
summary_df.to_csv(csv_name, index=False)

print("CSV report saved")
```

CSV report saved

PDF Report Generation

Initialize PDF and add a new page using FPDF

Set title: "Daily Sales Report"

Print summary values (Total Orders, Sales, Profit)

Display country-wise sales data

```
pdf_name = "sales_report_" + date + ".pdf"
```

Save PDF using pdf.output(pdf_name)

In [23]:

```
pdf = FPDF()
pdf.add_page()

pdf.set_font("Arial", size=14)
```

```

pdf.cell(200, 10, "Daily Sales Report", ln=True)

pdf.set_font("Arial", size=12)

for key, value in summary.items():
    pdf.cell(200, 10, f"{key}: {value}", ln=True)

pdf.cell(200, 10, "", ln=True)

pdf.cell(200, 10, "Sales by Country", ln=True)

for country, sales in country_sales.items():
    pdf.cell(200, 10, f"{country}: {round(sales, 2)}", ln=True)

pdf_name = f"sales_report_{date}.pdf"
pdf.output(pdf_name)

print("PDF report saved")

```

PDF report saved

Conclusion

The experiment successfully automated daily report generation from CSV data using Python. Data loading, cleaning, and summary analysis were performed efficiently without manual effort. Reports were generated and saved in both CSV and PDF formats for easy access and sharing. The use of task scheduling (cron/Task Scheduler) ensures the process runs automatically on a daily basis.

In []: